# SIMPLE AND EFFECTIVE SEMANTIC SONG SEGMENTATION

**Filip Korzeniowski*** **and Richard Vogl***
Music AI

## ABSTRACT

We propose a simple, yet effective approach to semantic song segmentation. Our model is a convolutional neural network trained to jointly predict frame-wise boundary activation functions and segment label probabilities. The input features consist of a log-magnitude log-frequency spectrogram and self-similarity lag matrices, combining modern deep learning approaches with hand-crafted features.

To evaluate our approach, we first examine commonly used datasets and find substantial overlap (up to 22%) between training and testing sets (SALAMI vs. RWC-Pop). As this overlap invalidates meaningful comparisons, we propose using the previously unexplored McGill Billboard dataset for testing. We carefully eliminate duplicate entries between McGill Billboard and other datasets through both audio fingerprinting and string-matching of song titles and artist names. Using the resulting set of 719 tracks, we demonstrate the effectiveness of our approach.

## 1. INTRODUCTION

Music Structure Analysis (MSA) is the task of dividing a piece of music into nonoverlapping segments that correspond to a human's analysis or perception of the structure of the piece. Such segments can optionally be labeled semantically (for example, *exposition*, *solo*, *chorus*, *intro*, etc.), by similarity ($A$, $B$, $B'$, etc.) or both. They can also be subdivided into finer subsegments, forming a hierarchical segmentation of a piece [1].

Historically, MSA relied on hand-crafted features and careful design of segment or boundary detection algorithms. These include methods that detect changes in diagonals of self-similarity matrices [2, 3], hidden Markov models [4], matrix factorization [5] or clustering algorithms [6, 7].

Recently, research has focused on models that detect segment boundaries and predict segment labels from a fixed vocabulary using deep learning. Such approaches often out-perform classical methods but require large amounts of data to train on, e.g., datasets such as SALAMI [8] and Harmonix [9]. To further expand the available training data, researchers explored unsupervised [10, 11] and semi-supervised [12] training, as well as partially labeled data [13] to improve their models.

Work in music segmentation has also explored strategies that leverage metric learning. In particular, Salamon et al. [14] introduce an approach that replaces handcrafted features with deep audio embeddings learned via few-shot and auto-tagging frameworks. While such metric learning–based techniques are attractive because they reduce annotation requirements and open the door to unsupervised training, they usually perform sub-par compared with supervised methods.

In this work, we focus on *semantic segmentation* of audio recordings of mostly Western music. For simplicity, we consider only a single level of boundary annotations and a simple, flat taxonomy of high-level functional segment labels, such as *verse*, *chorus*, and *bridge*. We rely only on supervised learning and consider scaling up through self-supervised learning or partially labeled data as orthogonal avenues to explore in the future.

Furthermore, we examine the evaluation protocols used in previous work and highlight three issues: first, datasets often used for "cross-dataset" evaluation overlap significantly; second, papers often do not specify whether they used "trimmed" metrics or not, and we found instances where invalid comparisons have been made due to this factor; third, models are typically trained using different dataset and dataset sizes, creating a confounding factor besides the method itself. These issues invalidate clean, direct comparisons between methods.

Our contributions can be summarized as:

- We propose a simple, yet effective approach to semantic song segmentation which surpasses state-of-the-art results on a variety of datasets.
- We identify and review issues in evaluation related to both metrics and data, and suggest ways to overcome some of them.
- We propose to use a dataset yet unexplored for structural segmentation as unseen test set: the *McGill Billboard* dataset.

The remainder of the work is structured as follows: First we discuss techniques in previous works relevant for this task in Section 2. In Section 3 we explain our approach to MSA in detail, while in Section 4 we discuss the used datasets, evaluation metrics, and the experimental setup. Finally, we present and discuss our results in Section 5 and conclude the paper in Section 6.

* Equal contribution.

© F. Korzeniowski and R. Vogl. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** F. Korzeniowski and R. Vogl, "Simple and Effective Semantic Song Segmentation", in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.,* Daejeon, South Korea, 2025.

## 2. RELATED WORK

Ullrich et al. [15] pioneered supervised training of deep learning models for boundary detection, improving the state-of-the-art detection F1 score by 40%. They also introduced a Gaussian loss weighting scheme to account for annotation inaccuracy and oversampling to mitigate scarceness of positive examples. Building upon this model, [16] proposed using self-similarity lag matrices as additional input legs to the network, separating the input spectrogram into harmonic and percussive parts, as well as training the model using multiple sources and levels of ground truth available in the SALAMI dataset.

Wang et al. [17] proposed adding a classification head for chorus detection, which are trained jointly with segment boundary prediction on top of the same deep learning backbone. They also introduced Hann-window smoothing to account for annotation inaccuracy and widened positive boundary targets to 0.5s. Following up on their work, in [18], they extended the chorus detection head to a larger set of section labels, used SpecTNT [19] as a backbone, and adopted the Connectionist Temporal Localization (CTL) loss [20] as an additional training objective.

Kim et al. [21] argue that MSA and beat-tracking are connected tasks and propose a model that jointly predicts beats, downbeats, segment boundaries, and labels. Their model requires a stem separation model for preprocessing to split the input audio into four stems. Then, it processes them using a convolutional front-end followed by transformer blocks with dilated neighborhood attention to capture temporal and inter-instrument dependencies. Finally, four classification heads predict the presence of (down-)beats and segments. Among other optimization tricks, they employ stochastic weight averaging (SWA) [22], which tends to improve the generalization of a model by searching for wider local optima.

Chen et al. [23] introduce a transformer-in-transformer model inspired by SpecTNT, aiming to analyze both spectral and long-term temporal dependencies. Their model features alternating spectral and temporal encoder layers with specialized multi-head self-attention (MHSA) mechanisms, tailored to deal with the specificities of spectral and temporal data. They claim improved performance on three datasets, enabled by the improved ability of the system to analyze non-local dependencies in music.

Buisson et al. [24] redefine structure analysis as a pairwise link prediction problem, where the system learns to classify pairs of beats as belonging to the same structural segment or not. Their method integrates graph attention networks (GATs) to combine link and node features, leveraging a self-similarity matrix to capture temporal dependencies. The model is further refined through MinCut regularization and a multi-task training objective, enabling it to predict both segment boundaries and section labels.

## 3. METHOD

We craft our segmentation model by integrating ideas from previous work into a new model and training recipe. Our aim is to create a lightweight model that is able to efficiently process large amounts of data while minimizing computational cost and memory footprint. To achieve this, we deviate in two ways from current trends in deep learning (which tend to produce ever-so-larger end-to-end models): first, we employ temporal convolutional networks (TCN) instead of transformer-based models; secondly, we resort to meaningful, hand-crafted features as additional input instead of relying on increased model complexity to learn such features. See Fig. 1 for a model overview.

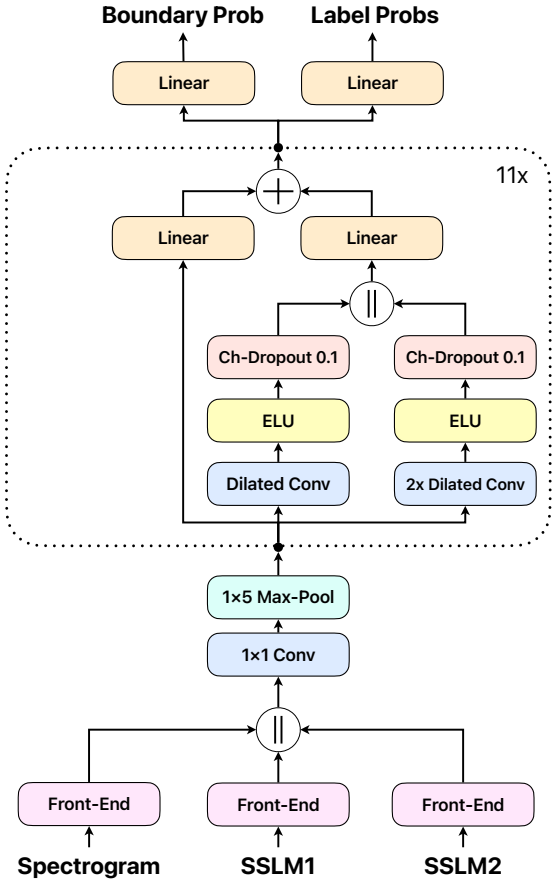



**Figure 1**. Model Overview. Three identical convolutional front-end networks (c.f. Table 1) pre-process each of the input features. The concatenated result is down-sampled in time by 5 ($1 \times 5$ max-pooling) and then feed into a TCN stack. The TCN stack consists of 11 blocks of two parallel dilated convolutions with increasing dilation rates, ELU activations, dropout and a residual connection. The dilation rates start at 1 and 2 for the parallel convolutions and are doubled for each of the 11 blocks.

### 3.1 Input Features

#### *3.1.1 Spectrogram*

The main input to the model is, similar to [25], a log-frequency log-magnitude spectrogram. The magnitude spectrogram is computed from 44.1kHz audio using Hann-windows of size 2048 at 100 frames per second (fps). We

apply a filterbank of logarithmically spaced triangular filters with 12 bands per octave between 30Hz and 17kHz, resulting in $F = 81$ frequency bands. Finally, the natural logarithm (adding $\epsilon = 1e^{-6}$ for numerical stability) is applied to compress the magnitude, resulting in a spectrogram denoted as $x_t$ for $t = 1 \dots T$.

#### *3.1.2 Self-Similarity Lag Matrices*

From this spectrogram, we extract two self-similarity lag matrices (SSLMs), similar as described in [16]. First, the time axis is down-sampled by a factor of four to 25 fps, using max-pooling (pooling factor $p = 4$, $T' = T/p$). Then, the discrete cosine transform (DCT) is applied frame-wise, discarding the DC coefficient (first bin). This results in a MFCC-like representation of the signal.

Next, frames of this representation within a temporal context of $\pm C$ ($C = 2$ equaling 0.08s of audio at 25 fps, 5 frames total) are stacked, to build overlapping feature blocks $\hat{x}_t$. Using these feature blocks, the cosine distance $d_{\cos}$ is calculated between feature blocks up to a lag $L$ of 90 seconds ($L = 2250$ frames at 25 fps), resulting in a distance matrix of size $T' \times L$.

$$D_{t,l} = d_{\cos}\left(\hat{x}_t, \hat{x}_{t-l}\right), \quad t = 1 \dots T', \quad l = 1 \dots L. \tag{1}$$

The resulting matrix is then normalized using an adaptive threshold $\epsilon_{t,l}$, the mean of the quantile $Q_\kappa$ with $\kappa = 0.1$ of the distances in rows $t$ and $t - l$ of $D$:

$$\epsilon_{t,l} = \overline{Q_\kappa}\left(D_{t,1}, \dots, D_{t,L}, D_{t-l,1}, \dots, D_{t-l,L}\right). \tag{2}$$

The input signal is considered time-circular, thus indices $(t - l) < 1$ are wrapped around to $t' = (t - l) + T'$.

After normalization, the sigmoid function $\sigma(\cdot)$ is applied for smoothing, resulting in the final relationship matrix:

$$R_{t,l} = \sigma\left(1 - \frac{D_{t,l}}{\epsilon_{t,l}}\right). \tag{3}$$

For the two different SSLM matrices, the relationship matrix is down-sampled in feature dimension by max-pooling. The first one is using a kernel size of 6, capturing local similarity, the other a kernel size of 22, capturing longer-term dependencies. The final number of feature values is limited to the closest 100 values. Both SSLM matrices are up-sampled again by a factor of four in time dimension using bicubic interpolation, to match the original 100 fps.

The two SSLMs and the filtered spectrogram represent the three features used as inputs for the convolutional front-end. While for some experiments in [16], an additional harmonic percussive separation is applied to the spectrogram, we found that this does not further improve performance. All features share the same time resolution, which simplifies further processing.

### 3.2 Convolutional Front-End

Each of the three input features is processed by a separate but identical front-end module similar to those proposed in [26]: three convolutions with 20 kernels of size $(3 \times 3)$, $(1 \times 10)$, and $(3 \times 3)$, respectively, followed by an ELU activation [27], element-wise dropout [28] with probability 0.1, and finally, $(3 \times 1)$ frequency-wise max-pooling. After each convolution, padding along the time axis is applied to maintain its full length, while the feature dimension is reduced to 1 by convolutions and max-pooling. See Table 1 for a concise summary of the front-end modules.

Concatenating the three 20-channel-outputs dimension results in 60 channels which are subsequently reduced to 30 using a $(1 \times 1)$ convolution. The time-axis is then down-sampled by a factor of 5 using max-pooling. This reduces the frame rate from 100 to 20 fps—sufficient for precise boundary detection, while reducing the computational load of the network's back-bone.

| **Convolutional Frontend** | |
|---|---|
| 2D Conv. | $(3 \times 3)$, 20 channels w/ ELU |
| Padding | $(0, 1)$ |
| Max-Pooling | $(3 \times 1)$, 0.1 dropout |
| Conv. Kernel | $(10 \times 1)$, 20 channels w/ ELU |
| Max-Pooling | $(3 \times 1)$, 0.1 dropout |
| Conv. Kernel | $(3 \times 3)$, 20 channels w/ ELU |
| Padding | $(0, 1)$ |
| Max-Pooling | $(3 \times 1)$, 0.1 dropout |

**Table 1**. Convolutional front-end configuration. Each block consists of a 2D convolution layer with ELU activation followed by max-pooling and dropout. No batch or other normalization is used. The time dimension (last) is always padded to keep its size, while the frequency dimension is reduced from 81 to size 1.

### 3.3 TCN Backbone

The combined features are processed through 11 sequential 1D TCN blocks (as described in [29]). Each block contains two parallel dilated 1D-convolutional layers, each with 30 kernels of size 5. The dilation rate of the second convolutional layer is twice that of the first, and these rates double progressively across blocks, starting from one. The dilated convolutions are followed by channel-wise dropout with probability 0.1 and an ELU activation function. Within each block, the outputs of both convolutions are concatenated and reduced from 60 to 30 channels using a linear projection (implemented by a 1D-convolution with kernel size 1). A residual connection, processed through another linear projection, is added to the final output of the block.

### 3.4 Multi-Task Output

The output of the backbone network is passed to two separate output heads. The output heads consist of a single linear layer followed by a sigmoid activation. Each of them projects the 30-dimensional input into a single boundary probability and eight probabilities corresponding to possible segment labels, respectively. [1] Using these output

[1] In theory, the segment label output should be a single categorical probability distribution, as labels are mutually exclusive; however, we found in preliminary experiments that individual probabilities for each segment label work better in practice.

heads, we obtain boundary and label probabilities at 20 frames per second.

### 3.5 Post-Processing

Finally, given the the probability timeseries, we must determine the concrete segment boundaries and assign segment labels. For segment boundaries, we use the peak-picking method introduced in [15]. We empirically optimize the detection threshold as well as the lengths of average- and max-filters used for peak picking on the validation set, and choose the following values: a symmetric 6s max filter, 16s pre-avg-filter, 8s post-avg filter, and a detection threshold of 0.2. To choose the segment labels, we follow [18] and select the label with the highest average probability within segment boundaries.

### 3.6 Training Setup

We train the model for a total of 100 epochs (each epoch is defined as 1000 updates) using stochastic gradient descent with a batch size of 1. We apply look-ahead optimization [30] with the RAdam update rule [31] using an initial learning rate of 0.002, which is divided by 5 after 60 epochs. We also clip gradients at a norm of 0.5.

To improve the generalization of the final model, we use stochastic weight averaging [22]. Starting from epoch 70, we increase the learning rate over 10 epochs to 0.001 using a cosine schedule and continue training for another 20 epochs. The final model weights are the average weights observed during the last 30 epochs.

### 3.7 Training Criterion

We employ binary cross-entropy as the loss function for both segment labels and boundaries. Given the scarcity of segment boundaries, we enhance training by smoothing the frame-wise targets over time with an exponential kernel of size 3 (0.15s). Additionally, we weight positive targets by a factor of 2.

We also observed that segment labels provide a stronger learning signal than segment boundaries. Consequently, simply summing the losses results in lower boundary detection accuracy. To address this, we apply a weight of 15 to the boundary loss. This adjustment ensures effective boundary detection with minimal impact on label accuracy.

Finally, to deal with double annotations in the SALAMI dataset, we follow [16] and use both annotation variants per track by duplicating the audio and associating it with one of the annotation variants each. Both samples are used in each epoch.

## 4. EXPERIMENT SETUP

### 4.1 Training & Evaluation Data

A variety of datasets have been used for training and evaluating structural segmentation systems. Among the most popular datasets are *Beatles* [32], *Harmonix* [9], *RWC* [33], *SALAMI* [8], and *Queen* [32].

These datasets were gathered over extended periods by independent research groups. As a result, they were annotated using diverse and often non-transparent guidelines. For example, while boundaries are aligned with downbeats in the Harmonix dataset, no such alignment is evident in SALAMI [34]. Similarly, the definitions of segment labels can vary between datasets. Consequently, some datasets may arguably represent different tasks (e.g. "find downbeat-aligned segment boundaries" vs. "find segment boundaries whenever a new phrase starts").

Additionally, some datasets overlap significantly. For instance, SALAMI was designed to contain tracks from both RWC and Isophonics. As a result, it includes 22 of the 100 tracks in RWC-Pop, as well as 21 out of 180 songs in the Beatles dataset.

All of this raises concerns about the reliability of cross-dataset evaluation, which is intended to assess a model's ability to generalize. However, if training and test sets overlap significantly, key assumptions of machine learning theory are compromised, and thus, evaluations become invalid. Similarly, if the test set represents a different task, the results do not reflect generalization to different music.

In this paper, we aim at building a model that works well for a wide variety of western-style pop music and evaluate it in a methodologically correct way. To this end, we employ 8-fold cross-validation on a mix of all the datasets mentioned above. To prevent data bleeding between test and training sets, we remove duplicates in the data, keeping the annotations of smaller datasets (e.g., we remove RWC and Beatles songs from Salami). We share our cross-validation partitions, annotations and model predictaions online [2], in order to facilitate more rigorous and reliable evaluations in future research.

For training, we use the original annotations from the Beatles dataset. Previous works typically rely on the "corrected" versions provided by TU Tampere and UPF. However, upon qualitative examination, we did not find these annotations to be "better" than the original ones (often to the contrary). This is supported by the results shown in Sec. 5, where evaluation using the original annotations typically yield higher scores, which indicates that the original annotations are more predictable (and thus, one may argue, more coherent) than the TUT ones.

### 4.2 Hold-Out Test Set

For comparison with existing work, we propose to use a dataset that has not been featured in the context of music segmentation at all: the *McGill Billboard* dataset [35]. Although this dataset has seen widespread usage for chord recognition and key identification, its structural segmentation annotations have not been utilized for training or evaluation. We remove duplicate tracks and eliminate overlap with the datasets mentioned previously using string-matching and audio fingerprinting methods, resulting in 719 tracks for testing.

[2] https://github.com/fdlm/ismir2025

| | Keyword | Label | | Keyword | Label |
|---|---|---|---|---|---|
| 1 | silence | silence | 15 | solo | solo |
| 2 | prechorus | verse | 16 | pre-chorus | verse |
| 3 | chorus | chorus | 17 | refrain | chorus |
| 4 | stutter | chorus | 18 | theme | chorus |
| 5 | rap | verse | 19 | verse | verse |
| 6 | slow | verse | 20 | section | verse |
| 7 | dialog | verse | 21 | build | verse |
| 8 | fadein | intro | 22 | intro | intro |
| 9 | bridge | bridge | 23 | opening | intro |
| 10 | out | outro | 24 | trans | bridge |
| 11 | ending | outro | 25 | coda | outro |
| 12 | inst | inst | 26 | break | inst |
| 13 | impro | inst | 27 | interlude | inst |
| 14 | end | silence | 28 | guitars | inst |
| | | | 29 | | inst |

**Table 2**. Segment label normalization. If an incoming label contains the keyword, it is mapped to the label provided in the second column. The order of the list indicates priority. Compared to the method from [18], we introduced a "solo" label, and added more mappings.

### 4.3 Evaluation Metrics

#### *4.3.1 Segment Boundaries*

We use a subset of the default metrics for semantic segmentation, computed using `mir_eval` [36]. For boundaries, we use F1 score with a tolerance of 0.5s. We opt for the *trimmed* version, i.e. we ignore the first and last "boundaries", which correspond to the beginning and end times of a track. These "boundaries" are semantically void and, to our belief, should not be evaluated, as they inflate detection results by up to 9% in F1 score (see Table 3).

Unfortunately, there is no clear consensus in previous work on whether to use trimmed or un-trimmed metrics to report results. Notably, the default parameters in the `mir_eval` library disable trimming, and therefore any paper that uses the library with default values reports inflated metrics. To complicate things further, many papers do not even report whether they used trimmed metrics or not. To ensure the best comparability in this work, we indicate if trimmed metrics were used by reference papers, and if not specified in the original paper, we examined the source code (if available) or contacted the authors to obtain that information.

#### *4.3.2 Segment Labels*

For segment labels, we use normalized conditional entropy (NCE), and label accuracy. For the latter, we compute the total duration of correct label predictions within a track and divide it by the length of the track. We extend the label vocabulary proposed in [18] by a *solo* label that indicates instrumental solos, in contrast to generic instrumental sections, resulting in an 8-class vocabulary consisting of *bridge*, *chorus*, *inst*, *intro*, *outro*, *silence*, *solo*, and *verse*. We also extend the normalization of the original labels by introducing new mappings to the method presented in [18], see Table 2.

#### *4.3.3 Multiple Annotations*

The SALAMI dataset includes two human-generated annotations for most tracks. Because both are human-labeled, both should be regarded as "correct" segmentations according to the task definition. Therefore, if a model accurately predicts either annotation, it should achieve the highest possible score for that track, regardless of any disagreement with the other annotation. To capture this, we evaluate model predictions against all annotations, group results by track, and report the highest score for each metric. [3]

## 5. RESULTS AND DISCUSSION

### 5.1 Overall Results

Table 3 presents the overall results across all datasets used for cross-validation in this study (except Queen due to its limited size). As indicated earlier, there are some caveats to consider when interpreting these results. These caveats are not specific to this study, but for transparency we deliberately want to draw attention to them.

First, most papers use different training sets and setups for reporting the results. For example, in the case of Harmonix, although most studies perform cross-validation, they use a different number of partitions (4 in [13, 18], 8 in others) and/or include additional training data like in [13, 18] and our work, while others only use Harmonix itself. The effect of using additional training data is not straightforward to assess: on the one hand, additional data for training often improves generalization and performance of a model; on the other hand, including other datasets might shift the data distribution in a way that reduces results for one particular dataset, especially if annotation guidelines or genre distributions vary. Second, as mentioned in previous sections, some datasets overlap; this can indicate that even in "cross-dataset" evaluations—which purport to indicate the generalization of a model—there might be train-test overlap, which could lead to inflated results. Finally, evaluation methods may differ subtly in their implementation or the hyper-parameters used. We already discussed the example of F1 boundary hit rate, where trimming has a huge impact on the absolute results.

In Tab. 3, we indicate which setup was used by each method (see the table caption for details). We also indicate which results were taken directly from the original paper, and which were re-produced using publicly available inference code and models and our own evaluation code (which is based on `mir_eval`). We also denote for which comparisons there is a train/test overlap which may inflate results. While we believe that these measures increase transparency, they do not solve the underlying issues which lead to limited comparability in the first place.

With all the caveats in mind, we can still identify tendencies in the metrics. Our proposed model performs best in detecting boundaries for all datasets except RWC-Pop, and exhibits the highest label prediction accuracy. Notably, in terms of NCE, LinkSeg [24] often out-performs our

[3] For reference, using the *average* instead of *max* reduces F1 by 5%.

| | Setup | F1 (tr) | F1❶ | NCE | Acc |
|---|---|---|---|---|---|
| *Harmonix — 911 tracks* | | | | | |
| [16] G&S ⊙ | CD | 0.578 | 0.644 | 0.717 | - |
| [18] SpecTNT | CV-4+ | - | 0.570 | 0.714 | 0.701 |
| [13] MuSFA | CV-4+ | - | 0.595 | - | 0.714 |
| [24] LinkSeg | CV-8 | | - | 0.772 | 0.742 |
| [21] All-In-1 | CV-8 | - | 0.660 | 0.769 | - |
| [21] All-In-1 ⊙ | CV-8 | 0.583 | 0.646 | 0.740 | 0.729 |
| **Proposed** | CV-8+ | **0.630** | 0.682 | **0.790** | **0.773** |
| *RWC-Pop — 100 tracks* | | | | | |
| [16] G&S ⊙ | CD⚠ | 0.507 | 0.571 | 0.744 | - |
| [18] SpecTNT | CD⚠ | - | 0.623 | 0.728 | 0.675 |
| [13] MuSFA | CD⚠ | - | 0.643 | - | 0.677 |
| [24] LinkSeg | CD | **0.648** | - | **0.812** | 0.747 |
| [23] MF-Sim | CD⚠ | - | 0.570 | - | 0.589 |
| [21] All-In-1 ⊙ | CD | 0.557 | 0.613 | 0.727 | 0.720 |
| **Proposed** | CV-8+ | 0.557 | 0.608 | 0.729 | **0.770** |
| *Beatles (TUT) — 174 tracks* | | | | | |
| [16] G&S ⊙ | CD⚠ | 0.457 | 0.566 | 0.659 | - |
| [23] MF-Sim | CD⚠ | - | 0.521 | - | 0.495 |
| [24] LinkSeg ⊙ | CD | 0.463 | 0.559 | **0.747** | 0.495 |
| [21] All-In-1 ⊙ | CD | 0.437 | 0.549 | 0.639 | 0.439 |
| **Proposed** | CV-8 | **0.549** | 0.626 | 0.721 | **0.598** |
| *Beatles (Orig) — 180 tracks* | | | | | |
| [16] G&S ⊙ | CD⚠ | 0.550 | 0.639 | 0.674 | - |
| [24] LinkSeg ⊙ | CD | 0.467 | 0.562 | **0.741** | 0.485 |
| [21] All-In-1 ⊙ | CD | 0.455 | 0.563 | 0.637 | 0.450 |
| **Proposed** | CV-8 | **0.613** | 0.681 | 0.719 | **0.597** |
| *SALAMI-Pop (Clean) — 191 tracks* | | | | | |
| [18] SpecTNT | CD⚠ | - | 0.490 | 0.632 | 0.544 |
| [13] MuSFA | CD⚠ | - | 0.532 | - | 0.551 |
| [23] MF-Sim | CD⚠ | 0.505 | - | - | 0.497 |
| [24] LinkSeg ⊙ | CD | 0.503 | 0.584 | **0.743** | 0.575 |
| [21] All-In-1 ⊙ | CD | 0.507 | 0.596 | 0.700 | 0.545 |
| **Proposed** | CV-8+ | **0.607** | 0.674 | 0.731 | **0.682** |
| *SALAMI (Clean) — 1239 tracks* | | | | | |
| [24] LinkSeg ⊙ | CD | 0.413 | 0.494 | 0.694 | 0.467 |
| [21] All-In-1 ⊙ | CD | 0.415 | 0.507 | 0.659 | 0.426 |
| **Proposed** | CV-8+ | **0.555** | 0.632 | **0.720** | **0.614** |
| *SALAMI (Clean, G&S Test) — 452 tracks* | | | | | |
| [16] G&S ⊙ | TT | 0.519 | 0.603 | 0.664 | - |
| [24] LinkSeg ⊙ | CD | 0.412 | 0.488 | 0.703 | 0.479 |
| [21] All-In-1 ⊙ | CD | 0.407 | 0.496 | 0.673 | 0.462 |
| **Proposed** | CV-8+ | **0.554** | 0.628 | **0.732** | **0.629** |

**Table 3**. Overall results. *CD* indicates a cross-dataset setup, *CV-N* means cross-validation with N partitions, *TT* indicates a train-test split, + indicates usage of additional data, ⚠ indicates train/test overlap. Results of methods marked with ⊙ are calculated using original checkpoints and inference code. **F1❶** is shown for reference only, and **F1 (tr)** should be considered instead. For *SALAMI (Clean)*, we removed overlaps. *SALAMI (Clean, G&S Test)* is the intersection of SALAMI (Clean) and the test set used in [16].

| | F1 (tr) | NCE | Acc |
|---|---|---|---|
| [16] G&S | 0.569 | 0.678 | - |
| [24] LinkSeg | 0.461 | 0.752 | 0.629 |
| [21] All-In-1 | 0.491 | 0.683 | 0.590 |
| **Proposed** | **0.647** | **0.754** | **0.668** |

**Table 4**. Results on McGill Billboard dataset (719 tracks).

model, indicating that while our proposed method predicts the correct label more often, LinkSeg offers more *consistent* labeling (disregarding its semantic meaning). This indicates that the graph-link approach is better capable of identifying which sections are similar, even if the assigned label is incorrect.

Strikingly, LinkSeg outperforms every compared method on the RWC-Pop dataset by a large margin in boundary detection F1 and label NCE. A more thorough, qualitative examination of the predicted segments may be required to determine why their approach lends itself so well for this particular dataset.

### 5.2 McGill Billboard Results

To circumvent some of the issues discussed above, we propose to use the McGill Billboard dataset, which has not been used for neither evaluation nor training in any of the methods we compare against. Here, we can only compare methods for which we have access to the inference code and models. The results are shown in Tab. 4.

On this dataset, our proposed model clearly outperforms the compared methods. Interestingly, even the decade-old G&S model from [16] gives better results than the more modern All-In-1 and LinkSeg, mimicking similar tendencies seen in Tab. 3 for Harmonix (G&S > LinkSeg), Beatles (TUT) (G&S > All-In-1), Beatles (Orig) (G&S > LinkSeg and All-In-1), and Salami (Clean G&S Test Split) (G&S > LinkSeg and All-In-1).

## 6. CONCLUSION

In this paper, we introduced a simple, yet effective method for semantical song segmentation based on a multi-leg TCN architecture that combine a raw log-frequency log-magnitude spectrogram and hand-crafted self-similarity lag matrices to predict segment boundaries and labels.

We identified, explored, and proposed remedies to challenges in evaluation, most notably train/test overlap and inconsistent configurations for evaluation metrics. With this in mind, we evaluated and compared our model against state-of-the-art methods on a wide variety of datasets. We also attempted to provide a cleaner comparison by proposing to use the hitherto unused McGill Billboard dataset as test set, for which we eliminated overlaps with existing datasets used for training. In all scenarios, our method yields superior performance on most datasets.

Future work could explore the addition of link-prediction methods like the ones used in [24], as they have shown to achieve promising results in terms of label prediction consistency. Also, scaling up data using partially labeled datasets may further improve results.

## 7. REFERENCES


[1] O. Nieto, G. J. Mysore, C.-i. Wang, J. B. L. Smith, J. Schlüter, T. Grill, and B. McFee, "Audio-Based Music Structure Analysis: Current Trends, Open Challenges, and Applications," *Transactions of the International Society for Music Information Retrieval*, vol. 3, no. 1, Dec. 2020.

[2] M. Müller, "Audio Structure Analysis," in *Information Retrieval for Music and Motion*, ser. SpringerLink: Springer e-Books. Berlin, Heidelberg: Springer-Verlag, 2007.

[3] J. Paulus, M. Müller, and A. Klapuri, "Audio-based Music Structure Analysis," in *Proceedings of the 11th International Conference on Music Information Retrieval (ISMIR)*, Utrecht, Netherlands, Sep. 2010.

[4] M. Levy and M. Sandler, "Structural Segmentation of Musical Audio by Constrained Clustering," *IEEE Trans. Audio Speech Lang. Process.*, vol. 16, no. 2, Feb. 2008.

[5] O. Nieto and T. Jehan, "Convex Non-Negative Matrix Factorization for Automatic Music Structure Identification," in *Proceedings of the 2013 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Vancouver, Canada, May 2013.

[6] O. Nieto and J. P. Bello, "Music Segment Similarity Using 2D-Fourier Magnitude Coefficients," in *Proceedings of the 2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Florence, Italy, May 2014.

[7] B. McFee and D. Ellis, "Analyzing Song Structure with Spectral Clustering." in *Proceedings of 15th International Society for Music Information Retrieval Conference (ISMIR)*, Taipei, Taiwan, Oct. 2014.

[8] J. B. L. Smith, J. A. Burgoyne, I. Fujinaga, D. D. Roure, and J. S. Downie, "Design and Creation of a Large-Scale Database of Structural Annotations," in *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR)*, Miami, USA, Oct. 2011.

[9] O. Nieto, M. McCallum, M. E. P. Davies, A. Robertson, A. Stark, and E. Egozy, "The HARMONIX Set: Beats, Downbeats, and Functional Segment Annotations of Western Popular Music," in *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*, Delft, The Netherlands, Nov. 2019.

[10] M. C. McCallum, "Unsupervised Learning of Deep Features for Music Segmentation," in *Proceedings of the 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Brighton, United Kingdom, May 2019.

[11] M. Buisson, B. McFee, S. Essid, and H. C. Crayencour, "A Repetition-Based Triplet Mining Approach for Music Segmentation," in *Proceedings of the 24th International Society for Music Information Retrieval Conference (ISMIR)*, Milan, Italy, Nov. 2023.

[12] Y.-N. Hung, J.-C. Wang, M. Won, and D. Le, "Scaling Up Music Information Retrieval Training with Semi-Supervised Learning," *arXiv*, vol. arXiv:2310.01353, Oct. 2023.

[13] J.-C. Wang, J. B. L. Smith, and Y.-N. Hung, "MuSFA: Improving Music Structural Function Analysis with Partially Labeled Data," in *Late-Breaking/Demo Session of the 23rd International Society for Music Information Retrieval Conference (ISMIR)*, Bengaluru, India, Dec. 2022.

[14] J. Salamon, O. Nieto, and N. J. Bryan, "Deep embeddings and section fusion improve music segmentation," in *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*, Online, Nov. 2021.

[15] K. Ullrich, J. Schlüter, and T. Grill, "Boundary Detection in Music Structure Analysis Using Convolutional Neural Networks," in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, Taipei, Taiwan, Oct. 2014.

[16] T. Grill and J. Schlüter, "Music Boundary Detection using Neural Networks On Combined Features and Two-Level Annotations," in *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR)*, Málaga, Spain, Oct. 2015.

[17] J.-C. Wang, J. B. L. Smith, J. Chen, X. Song, and Y. Wang, "Supervised Chorus Detection for Popular Music Using Convolutional Neural Network and Multi-Task Learning," in *Proceedings of the 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Toronto, Canada, Apr. 2021.

[18] J.-C. Wang, Y.-N. Hung, and J. B. L. Smith, "To Catch a Chorus, Verse, Intro, or Anything Else: Analyzing a Song with Structural Functions," in *Proceedings of the 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Singapore, May 2022.

[19] W.-T. Lu, J.-C. Wang, M. Won, K. Choi, and X. Song, "SpecTNT: A Time-Frequency Transformer for Music Audio," in *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*, Online, Nov. 2021.

[20] Y. Wang and F. Metze, "Connectionist Temporal Localization for Sound Event Detection with Sequential Labeling," in *Proceedings of the 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Brighton, United Kingdom, May 2019.

[21] T. Kim and J. Nam, "All-In-One Metrical and Functional Structure Analysis with Neighborhood Attentions on Demixed Audio," in *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, New Paltz, USA, Oct. 2023.

[22] P. Izmailov, D. Podoprikhin, T. Garipov, D. Vetrov, and A. G. Wilson, "Averaging Weights Leads to Wider Optima and Better Generalization," in *Proceedings of the 34th Conference on Uncertainty in Artificial Intelligence (UAI)*, Monterey, USA, Aug. 2018.

[23] T.-P. Chen and K. Yoshii, "Learning Multifaceted Self-Similarity over Time And Frequency for Music Structure Analysis," in *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR)*, San Francisco, USA, Nov. 2024.

[24] M. Buisson, B. McFee, and S. Essid, "Using Pairwise Link Prediction and Graph Attention Networks for Music Structure Analysis," in *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR)*, San Francisco, USA, Nov. 2024.

[25] M. E. P. Davies and S. Bock, "Temporal Convolutional Networks for Musical Audio Beat Tracking," in *Proceedings of the 2019 27th European Signal Processing Conference (EUSIPCO)*, A Coruña, Spain, Sep. 2019.

[26] S. Böck and M. E. P. Davies, "Deconstruct, Analyze, Reconstruct: How to Improve Tempo, Beat, and Downbeat Tracking," in *Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR)*, Montréal, Canada, Oct. 2020.

[27] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)," in *Proceedings of the International Conference on Learning Representations (ICLR)*, San Juan, Puerto Rico, May 2016.

[28] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *The Journal of Machine Learning Research*, vol. 15, no. 56, Jun. 2014.

[29] S. Böck, M. E. P. Davies, and P. Knees, "Multi-Task Learning Of Tempo And Beat: Learning One To Improve The Other," in *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*, Delft, The Netherlands, Nov. 2019.

[30] M. R. Zhang, J. Lucas, G. Hinton, and J. Ba, "Lookahead Optimizer: k steps forward, 1 step back," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, Vancouver, Canada, 2019.

[31] L. Liu, H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and J. Han, "On the Variance of the Adaptive Learning Rate and Beyond," in *Proceedings of the 8th International Conference On Learning Representations (ICLR)*, Addis Ababa, Ethiopia, Apr. 2020.

[32] M. Mauch, C. Cannam, M. Davies, S. Dixon, C. Harte, S. Kolozali, D. Tidhar, and M. Sandler, "OMRAS2 Metadata Project 2009," in *Late-Breaking/Demos Session of the 10th International Conference on Music Information Retrieval (ISMIR)*, Kobe, Japan, Oct. 2009.

[33] M. Goto, H. Hashiguchi, T. Nishimura, and R. Oka, "RWC Music Database: Popular, Classical and Jazz Music Databases," in *Proceedings of the 3rd International Conference on Music Information Retrieval (ISMIR)*, Paris, France, Oct. 2002.

[34] A. Marmoret, J. E. Cohen, and F. Bimbot, "Barwise Music Structure Analysis with the Correlation Block-Matching Segmentation Algorithm," *Transactions of the International Society for Music Information Retrieval*, vol. 6, no. 1, Nov. 2023.

[35] J. A. Burgoyne, J. Wild, and I. Fujinaga, "An Expert Ground Truth Set for Audio Chord Recognition and Music Analysis." in *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR)*, Miami, USA, Oct. 2011.

[36] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, and D. P. W. Ellis, "Mir_eval: A Transparent Implementation of Common MIR Metrics," in *Proceedings of the 15th International Conference on Music Information Retrieval (ISMIR)*, Taipei, Taiwan, Oct. 2014.